

Project 2 Writeups

1. WU-1

(A): Train depth 3 decision trees on the WineDataSmall task. What words are most indicative of *being* Sauvignon-Blanc? Which words are most indicative of not being Sauvignon-Blanc? What about Pinot-Noir (label==2)?

Answer:

1. Train and get accuracy:

```
h = OAA(5, lambda: DecisionTreeClassifier(max_depth=3))
h.train(WineDataSmall.X, WineDataSmall.Y)
P = h.predictAll(WineDataSmall.Xte)
accuracy = np.mean(P == WineDataSmall.Yte)
```

Accuracy = 0.5842450765864332

2. Indicative words

```
print("Decision tree for being Sauvignon-Blanc:")
util.showTree(h.f[0], WineDataSmall.words)
print("Decision tree for being Pinot-Noir:")
util.showTree(h.f[2], WineDataSmall.words)
```

Result:

Decision tree for being Sauvignon-Blanc:

```
Decision tree for being Sauvignon-Blanc:
citrus?
-N-> lime?
|   -N-> gooseberry?
|   |   -N-> class 0 (356 for class 0, 10 for class 1)
|   |   -Y-> class 1 (0 for class 0, 4 for class 1)
|   |   -Y-> hint?
|   |   |   -N-> class 1 (1 for class 0, 15 for class 1)
|   |   |   -Y-> class 0 (2 for class 0, 0 for class 1)
|   -Y-> grapefruit?
|   |   -N-> flavors?
|   |   |   -N-> class 1 (4 for class 0, 12 for class 1)
|   |   |   -Y-> class 0 (11 for class 0, 5 for class 1)
|   |   -Y-> lingering?
|   |   |   -N-> class 1 (0 for class 0, 14 for class 1)
|   |   |   -Y-> class 0 (1 for class 0, 0 for class 1)
Decision tree for being Pinot-Noir:
cherry?
-N-> raspberries?
|   -N-> strawberry?
|   |   -N-> class 0 (225 for class 0, 58 for class 1)
|   |   -Y-> class 1 (0 for class 0, 4 for class 1)
|   -Y-> ;?
|   |   -N-> class 1 (0 for class 0, 12 for class 1)
|   |   -Y-> class 0 (1 for class 0, 0 for class 1)
-N-> cassis?
|   -N-> verdot?
```

```
| | -N-> class 1 (36 for class 0, 68 for class 1)
| | -Y-> class 0 (8 for class 0, 0 for class 1)
| -Y-> allspice?
| | -N-> class 0 (21 for class 0, 0 for class 1)
| | -Y-> class 1 (0 for class 0, 2 for class 1)
```

Most indicative words for being Sauvignon-Blanc: hint, flavors, and lingering.

Most indicative words for not being Sauvignon-Blanc: gooseberry, lime, and grapefruit.

Most indicative words for being Pinot-Noir: strawberry.

(B) Train depth 3 decision trees on the full WineData task (with 20 labels). What accuracy do you get? How long does this take (in seconds)? One of my least favorite wines is Viognier -- what words are indicative of this?

Answer:

1. Train: accuracy and time

```
start_time = time.time()
h = multiclass.OAA(20, lambda: DecisionTreeClassifier(max_depth=3))
h.train(WineData.X, WineData.Y)
train_time = time.time() - start_time
P = h.predictAll(WineData.Xte)
accuracy = np.mean(P == WineData.Yte)
```

Train time = 0.3877554 seconds

Accuracy = 0.37012987012987014

2. Indicative words

```
util.showTree(h.f[17], WineData.words)
```

Result:

```
peaches?
-N-> nectarine?
| -N-> chilled?
| | -N-> class 0 (1036 for class 0, 1 for class 1)
| | -Y-> class 0 (6 for class 0, 1 for class 1)
| -Y-> 5?
| | -N-> class 0 (13 for class 0, 1 for class 1)
| | -Y-> class 1 (0 for class 0, 1 for class 1)
-Y-> milk?
| -N-> harmonious?
| | -N-> class 0 (14 for class 0, 0 for class 1)
| | -Y-> class 1 (0 for class 0, 1 for class 1)
| -Y-> class 1 (0 for class 0, 3 for class 1)
```

Indicative words for being Viognier: 5, harmonious, and milk.

(C) Compare the accuracy using zero-one predictions versus using confidence. How much difference does it make?

```
P_zero_one = h.predictAll(WineData.Xte, useZeroOne=True)
accuracy_zero_one = np.mean(P_zero_one == WineData.Yte)
P_confidence = h.predictAll(WineData.Xte, useZeroOne=False)
accuracy_confidence = np.mean(P_confidence == WineData.Yte)
```

Zero-one accuracy: 0.24489795918367346

P-confidence accuracy: 0.37012987012987014

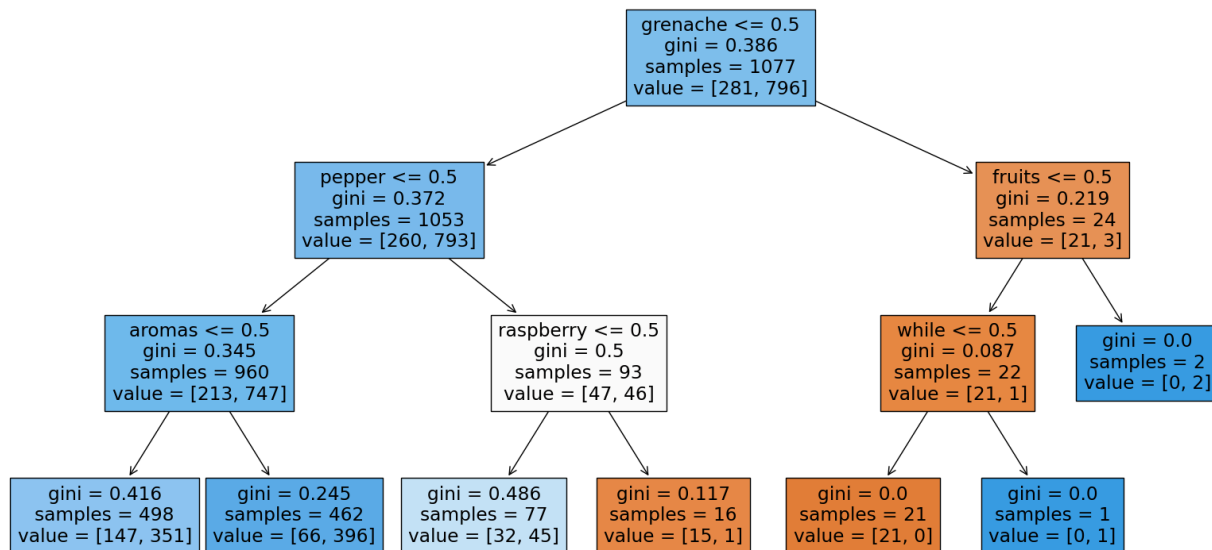
The P-confidence is about 1.5 times better than zero-one in this case with an extra ~12.5% accuracy.

2. WU-2

Show the test accuracy you get with a balanced tree on the WineData using a DecisionTreeClassifier with max depth 3.

Answer:

Decision tree of being Viognier:

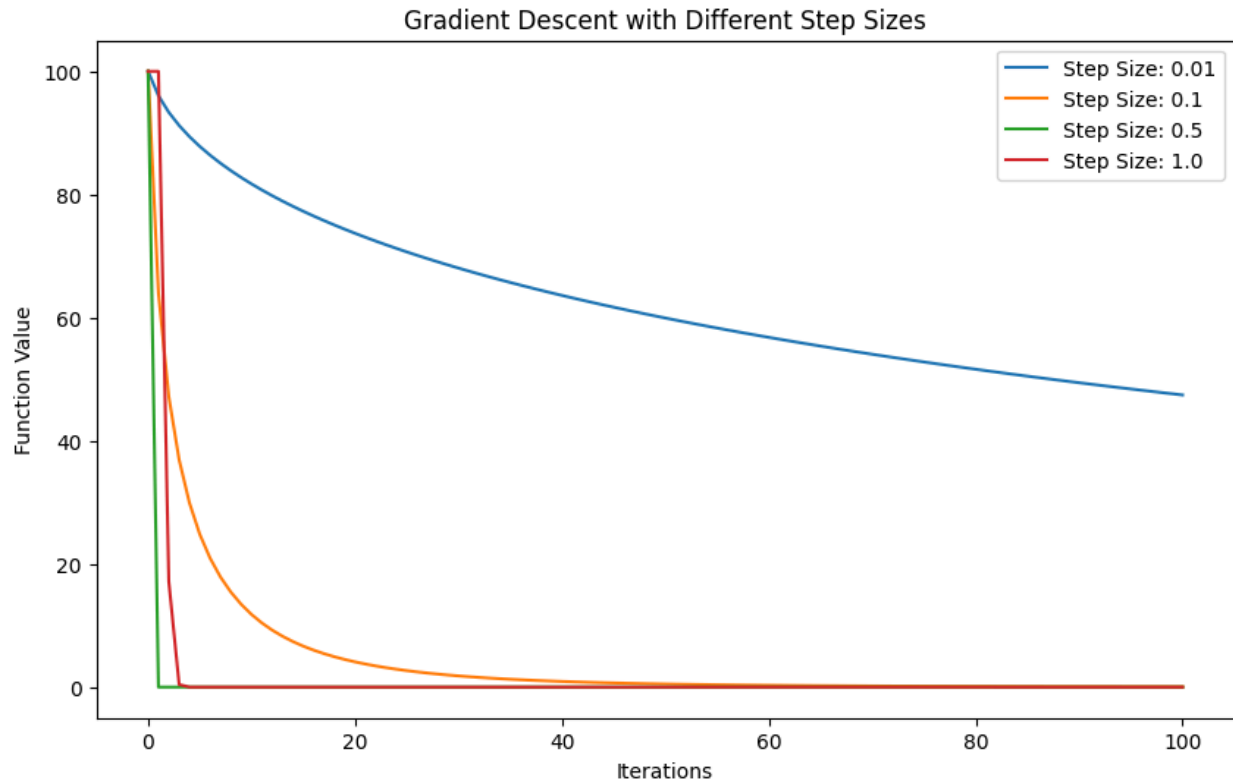


3. WU-3

What is the impact of the step size on convergence? Find values of the step size where the algorithm diverges and converges.

Answer:

Step Size: 0.01, Final x: 6.887781469736353, Final Function Value: 47.441533574843476
Step Size: 0.1, Final x: 0.21734427526984595, Final Function Value: 0.04723853399257457
Step Size: 0.5, Final x: 0.0, Final Function Value: 0.0
Step Size: 1.0, Final x: 0.0, Final Function Value: 0.0



The bigger the step size is, the faster it converges.

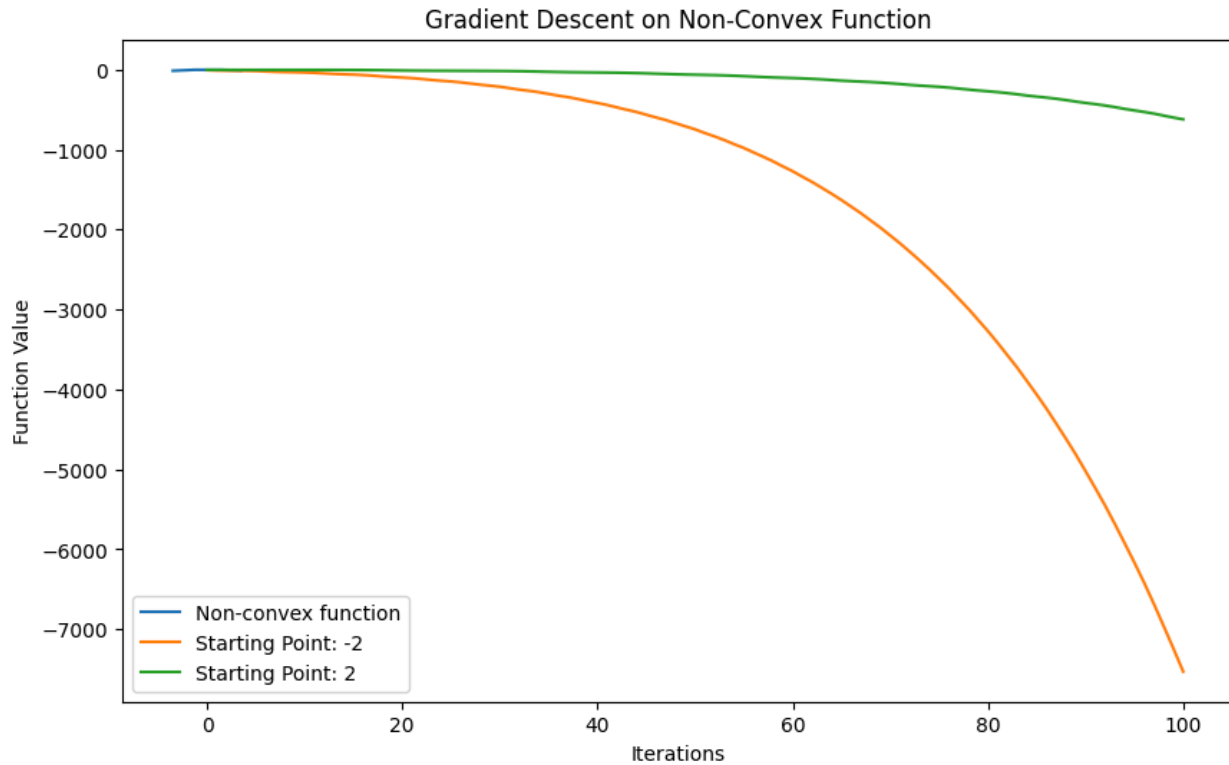
4. WU-4

Come up with a non-convex univariate optimization problem. Plot the function you're trying to minimize and show two runs of gd, one where it gets caught in a local minimum and one where it manages to make it to a global minimum. (Use different starting points to accomplish this.)"

Answer:

Functions:

```
fun1 = np.sin(3*x)-x**2+0.7*x
fun2 = 3*np.cos(3 * x)-2*x+0.7
starting_point1 = -2
starting_point2 = 2
```



5. WU-5:

For each of the loss functions, train a model on the binary version of the wine data (called WineDataBinary) and evaluate it on the test data. You should use $\lambda=1$ in all cases. Which works best? For that best model, look at the learned weights. Find the words corresponding to the weights with the greatest positive value and those with the greatest negative value. Hint: look at WineDataBinary.words to get the id-to-word mapping. List the top 5 positive and top 5 negative and explain.

Answer: